

2. Promesas

2.1. Promesas: introducción

La programación asíncrona tiene un objetivo fundamental: impedir que el programa principal se quede bloqueado esperando a un proceso que va a llevar cierto tiempo. Hay, por tanto, una **separación** en la ejecución del código: por un lado está el **lanzamiento** o arranque del proceso y por otro el **procesamiento del resultado**. En el proceso de lanzamiento es necesario definir qué función se encargará de realizar ese procesamiento. En el apartado anterior hemos visto que el uso de *callbacks* constituye una de las técnicas para ello: cuando se lanza el proceso, se le indica una función de *callback* que será ejecutada cuando finalice el proceso y que recibirá el resultado de dicho proceso para que pueda ser utilizado.

Hemos visto también que el uso de *callbacks* presenta una serie de inconvenientes: el *callback hell* y la no devolución de resultados (el código no se ajusta al paradigma funcional). Para resolver estos problemas se diseñaron las **promesas**.

Una **promesa** es un objeto que define una serie de **métodos especiales** y que **puede devolver** un resultado en el futuro. Una promesa puede encontrarse únicamente en uno de estos **tres estados**:

- **Pendiente** (**pending**). Es el estado inicial. Significa que el proceso está todavía en ejecución y que no ha terminado.
- **Resuelta** (**fulfilled**). Significa que el proceso ha terminado y ha generado un resultado.
- **Rechazada** (**rejected**). Significa que el proceso ha terminado pero ha generado un error.

! IMPORTANTE

Es muy importante entender que la **promesa no es el resultado** del proceso.

Los **métodos** que define una promesa son los siguientes:

- **Then**. Admite dos parámetros: una función que se ejecuta si la promesa se **resuelve** (cuyo parámetro es el **resultado**) y una función que se ejecuta si la promesa se **rechaza** (cuyo parámetro es el **error**). Normalmente se suele utilizar solo con el primer parámetro, ya que el error se suele procesar con **catch**.
- **catch**. Admite un parámetro: una función que se ejecuta si la promesa se **rechaza** (cuyo parámetro es el **error**).
- **finally**. Admite un parámetro: una función que se ejecutará, tanto si la promesa se **resuelve** como si se **rechaza**.

A continuación, se muestra un ejemplo de una función ficticia que devuelve una promesa:

```
// Lanzamiento del proceso asincrono
// IMPORTANTE: 'a' almacena una promesa, no el resultado del proceso.
let a = funcionQueDevuelveUnaPromesa(parametros);

// Procesamiento de los resultados
a.then(
  function(resultado) {
    // Aquí procesamos el resultado (si no hay error)
  },
  function(error) {
    // Aquí procesamos el error (si se produce)
  }
);

// Este código se ejecuta inmediatamente después de lanzar la promesa,
// antes de procesar su resultado o su error
console.log("Me muestro antes de que se resuelva la promesa");
```

Como curiosidad, podemos ver que las promesas utilizan a su vez funciones *callback* en sus métodos **then**, **catch** y **finally**. Lo que ocurre es que dichos *callbacks* están estandarizados: el primer *callback* del método **then** siempre es una función de un parámetro [el resultado]; el *callback* de la función **catch** siempre es una función de un parámetro [el error].

El código anterior también podría ser escrito de la siguiente manera:

```
// Lanzamiento del proceso asincrono
// IMPORTANTE: 'a' almacena una promesa, no el resultado del proceso.
let a = funcionQueDevuelveUnaPromesa(parametros);

// Procesamiento de los resultados
a.then(
  function(resultado) {
    // Aqui procesamos el resultado (si no hay error)
  }
).catch(
  function(error) {
    // Aqui procesamos el error (si se produce)
  });
```

Por último, se muestra a continuación un ejemplo con **finally**:

```
// Lanzamiento del proceso asincrono
// IMPORTANTE: 'a' almacena una promesa, no el resultado del proceso.
let a = funcionQueDevuelveUnaPromesa(parametros);

// Procesamiento de los resultados
a
  .finally(
    function() {
      // Este mensaje se muestra siempre, antes del resultado o error
      console.log('Primer finally');
    })
  .then(
    function(resultado) {
      // Aqui procesamos el resultado (si no hay error)
    }
  )
  .finally(
    function() {
      // Este mensaje se muestra siempre, después del resultado (si lo hay)
      // o antes del error (si lo hay)
      console.log('Segundo finally');
    })
  .catch(
    function(error) {
      // Aqui procesamos el error (si se produce)
    })
  .finally(
    function() {
```



```
// Este mensaje se muestra siempre, después del resultado o error
console.log('Tercer finally');
});
```

! IMPORTANTE

Se pueden encadenar varios **finally**. Al terminar, pasa la ejecución al siguiente método definido de la promesa.

2.2. Creación de promesas

JavaScript permite crear promesas mediante el constructor **Promise**. Dicho constructor admite dos parámetros: una función para lanzar cuando se resuelve y una función para lanzar cuando se rechaza.

```
let promesa = new Promise(function(resolver, rechazar) {
  // Código que debe llamar a la función 'resolver' o 'rechazar'
});
```

! IMPORTANTE

En el 95% de los casos no será necesario que crees promesas con este método. La mayoría de las veces, el trabajo con promesas se limita a utilizar funciones existentes que devuelven promesas, por lo que simplemente hay que trabajar con **then**, **catch** y **finally**.

Únicamente deberás construir tus propias promesas si quieres utilizar funciones o API que utilicen callbacks para convertirlas a promesas.

Por ejemplo, para convertir la función **setTimeout** a una promesa se podría utilizar un código como el siguiente:

```
function temporizador(tiempo) {
  // Devuelve una promesa
  return new Promise(function(resolver, rechazar) {
    // Se ejecuta un temporizador
    setTimeout(function() {
      // Cuando se agota el tiempo se resuelve la promesa
      // El resultado de la ejecución es una cadena de texto
      resolver(`Temporizador de ${tiempo} ms terminado`);
    }, tiempo);
  });
}

// Se lanzan dos promesas
let t1 = temporizador(4000);
let t2 = temporizador(6000);

t1.then((resultado) => {
  // Este mensaje se muestra cuando se resuelva la primera promesa
  // Es decir, 4 segundos después
  console.log(resultado);
});

t2.then((resultado) => {
```



```
// Este mensaje se muestra cuando se resuelva la primera promesa
// Es decir, 6 segundos después
console.log(resultado)
});

// El código que viene a continuación se ejecuta inmediatamente después de
lanzar las promesas
// Pero no espera a que las promesas se resuelvan
console.log("Este mensaje se muestra antes que los mensajes de los
temporizadores");
```

2.3. Encadenamiento de promesas

Las promesas pueden encadenarse, ya que el método **then** devuelve siempre una promesa; por tanto, es posible ejecutar:

```
let prom = miPromesa.then().then().then();
prom instanceof Promise; // true
```

La particularidad del código anterior es el valor devuelto por la función *callback* de **then**: es posible devolver un valor «convencional» o una promesa. Si la función utilizada por **then** devuelve una promesa, el siguiente **then** ejecutará su *callback* cuando se resuelva dicha promesa y obtendrá el **resultado** de su resolución; si, por el contrario, la función *callback* de **then** devuelve un valor que no sea una promesa, dicho valor se convierte a una promesa cuya resolución es el mismo valor. Por ejemplo:

```
miPromesa.then((res1) => {
    // 'res' es el resultado de la resolución de 'miPromesa'
    // Se devuelve una segunda promesa
    return miSegundaPromesa;
}).then((res2) => {
    // 'res2' es el resultado de la resolución de 'miSegundaPromesa'
    // Se devuelve un número
    return 50;
}).then((res3) => {
    // A pesar de que en el callback del anterior 'then' se devuelve un
    número,
    // es posible ejecutar de nuevo 'then' sobre el resultado
    // El número se convierte en una promesa cuyo valor de resolución es el
    mismo número
    console.log(res3); // 50
});;
```

! IMPORTANTE

El encadenamiento de promesas es diferente de ejecutar muchos **then** sobre la misma promesa.

Además, JavaScript define una serie de métodos para trabajar con conjuntos de promesas:

- **Promise.all**. Se utiliza para lanzar un conjunto de promesas a la vez, que se pasan en un *array*. Mediante **then**, espera a que se resuelvan **todas las promesas**. Devuelve un *array* con sus resultados. Si alguna de las promesas es rechazada, el error que genera es el error capturado, y el resto de resultados se ignoran.
- **Promise.race**. Se utiliza también para lanzar un conjunto de promesas a la vez, que se pasan en un *array*. La **primera** que se resuelva o rechace es la que genera el resultado [o error].

A continuación se muestra un ejemplo de `Promise.all`:

```
function temporizador(tiempo) {
  // Devuelve una promesa
  return new Promise(function(resolver, rechazar) {
    // Se ejecuta un temporizador
    setTimeout(function() {
      // Cuando se agota el tiempo, se resuelve la promesa
      // El resultado de la ejecución es una cadena de texto
      resolver(`Temporizador de ${tiempo} ms terminado`);
    }, tiempo);
  });
}

var t1 = temporizador(3000); // 3 segundos
var t2 = temporizador(4000); // 4 segundos
var t3 = temporizador(6000); // 6 segundos

Promise.all([t1, t2, t3]).then(function(resultados) {
  for (let res of resultados) {
    console.log(res);
  }
  // Cuando pasen 6 segundos se mostrarán los 3 mensajes a la vez
});
```

Y un ejemplo con `Promise.race`:

```
function temporizador(tiempo) {
  // Devuelve una promesa
  return new Promise(function(resolver, rechazar) {
    // Se ejecuta un temporizador
    setTimeout(function() {
      // Cuando se agota el tiempo, se resuelve la promesa
      // El resultado de la ejecución es una cadena de texto
      resolver(`Temporizador de ${tiempo} ms terminado`);
    }, tiempo);
  });
}

var t1 = temporizador(3000); // 3 segundos
var t2 = temporizador(4000); // 4 segundos
var t3 = temporizador(6000); // 6 segundos

Promise.race([t1, t2, t3]).then(function(resultado) {
  // Cuando pasen 3 segundos, se mostrará el mensaje del temporizador de
  3 segundos
  // El resto de resultados se descartan
  console.log(resultado);
});
```



2.4. Tratamiento de errores con promesas

Uno de los inconvenientes de los *callbacks* es que no es posible utilizarlos dentro de bloques `try...catch`. Por ejemplo, el siguiente código provoca un error que no es capturado:

```
try {
  setTimeout(function() {
    throw new Error("Error muy grave");
  }, 2000);
} catch (error) {
  // El error no se captura en este bloque
  console.log(error.message);
}

// En la consola aparece el siguiente mensaje:
// Uncaught Error: Error muy grave
```

El motivo es que, cuando se ejecuta el *callback* del temporizador, el bloque `try...catch` ya ha finalizado su ejecución. Esto hace que sea difícil procesar errores en código que contenga *callbacks*, ya que habría que poner un bloque `try...catch` dentro de cada *callback*.

Las promesas incorporan tratamiento de errores de serie: si se produce o se lanza un error dentro de un `then`, la promesa será rechazada y por tanto el error podrá capturarse por el manejador correspondiente. Por ejemplo:

```
function temporizador(tiempo) {
  return new Promise(function(resolver, rechazar) {
    setTimeout(function() {
      resolver(`Temporizador de ${tiempo} ms terminado`);
    }, tiempo);
  });
}

let t1 = temporizador(4000);

t1.then((resultado) => {
  // Lanzamos un error
  throw new Error("Error personalizado");

  // Este código no se llega a ejecutar
  console.log(resultado);
}).catch((error) => {
  // Se trata el error generado
  console.log(`Se ha producido un error con mensaje ${error.message}`);
});
```